

A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a neural network diagram.

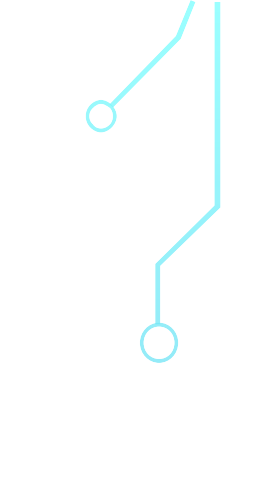
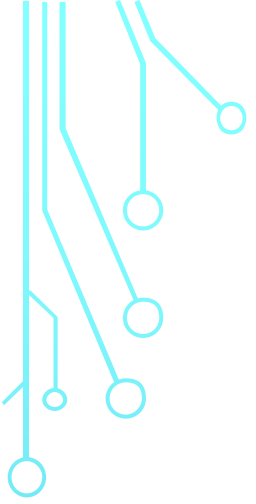
Bitwise Operators

COMP211 Connor McMahon

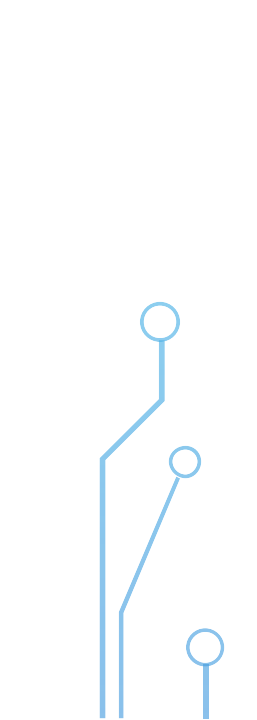
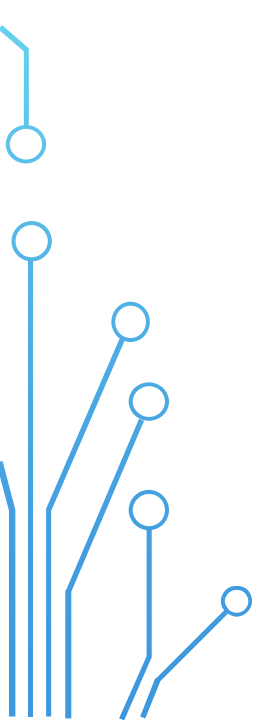
Masks

Masks

- A **mask** is a binary pattern used to **select (get), set, clear, or toggle specific bits** within a number.
- **Select**(get) = read
- **Set** = make 1
- **Clear** = make 0
- **Toggle** = flip from 0 to 1 or 1 to 0



Clear



clear_bit2

- Let's write a function, `clear_bit2`, that takes an integer as input, sets bit 2 to 0, and returns the modified result.
- First, let's write some test cases:

```
#define NUM_TESTS 3

int8_t clear_bit2(int8_t number);

int main() {
    int8_t tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n",
            tests[i], clear_bit2(tests[i]));
    }
    return 0;
}

...
```

14 = 0b0000_1110	➡	0b0000_1010 = 10
12 = 0b0000_1100		0b0000_1000 = 8
8 = 0b0000_1000		0b0000_1000 = 8

Program Output:

```
Original: 14, Modified: 10
Original: 12, Modified: 8
Original: 8, Modified: 8
```

clear_bit2

```
#define NUM_TESTS 3

int8_t clear_bit2(int8_t number);

int main() {
    int8_t tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit2(tests[i]));
    }
    return 0;
}

int8_t clear_bit2(int8_t number) {
    int8_t mask = ~(0b100);
    return number & mask;
}
```

clear_bit2 Example

```
int8_t clear_bit2(int8_t number) {  
    int8_t mask = ~(0b100);  
    return number & mask;  
}
```

number

0b0000_1111

mask

~(0b100)

~(0b0000_0100)

0b1111_1011

number & mask

	0b0000_1111
&	0b1111_1011
	0b0000_1011

clear_bit2 Example

```
int8_t clear_bit2(int8_t number) {  
    int8_t mask = ~(0b100);  
    return number & mask;  
}
```

number

0b1010_1010

mask

~(0b100)

~(0b0000_0100)

0b1111_1011

number & mask

	0b1010_1010
&	0b1111_1011
	0b1010_1010

clear_bit

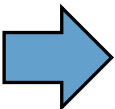
- Let's write a function that will clear bit *i* of the input *num*

```
#define NUM_TESTS 3

int8_t clear_bit(int8_t num, int8_t i);

int main() {
    int tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));
    }
    return 0;
}
```

Program Output:

14 = 0b0000_1110		0b0000_1010 = 10
12 = 0b0000_1100		0b0000_0100 = 8
8 = 0b0000_1000		0b0000_1000 = 8

```
Original: 14, Modified: 10
Original: 12, Modified: 4
Original: 8, Modified: 8
```

clear_bit

- Let's write a function that will clear bit *i* of the input *num*

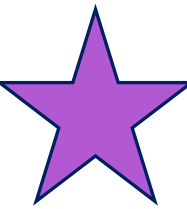
```
#define NUM_TESTS 3

int8_t clear_bit(int8_t num, int8_t i) {

}

int main() {
    int tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));
    }
    return 0;
}
```

clear_bit



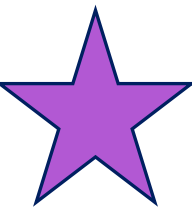
- Let's write a function that will clear bit `i` of the input `num`

```
#define NUM_TESTS 3

int8_t clear_bit(int8_t num, int8_t i) {
    int8_t mask = ~(1 << i);
    return num & mask;
}

int main() {
    int tests[NUM_TESTS] = {14, 12, 8};
    for (int i = 0; i < NUM_TESTS; i++) {
        printf("Original: %d, Modified: %d\n", tests[i], clear_bit(tests[i], i + 2));
    }
    return 0;
}
```

clear_bit Example



```
int8_t clear_bit(int8_t num, int8_t i) {  
    int8_t mask = ~(1 << i);  
    return num & mask;  
}
```

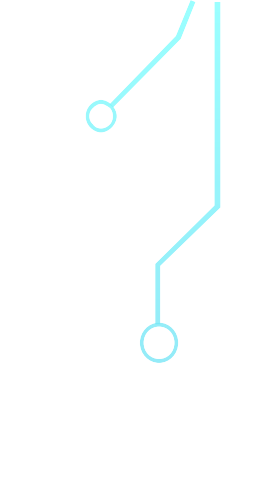
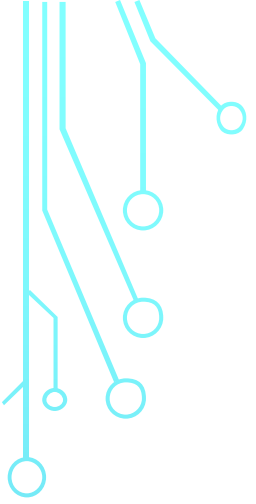
i=5 number
 0b1110_1010

 mask
 ~(1<<5)
~(0b0010_0000)
0b1101_1111

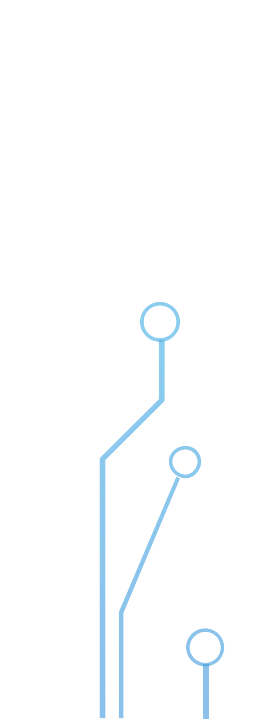
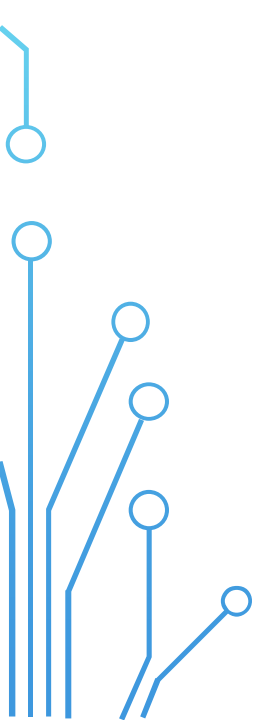
number & mask

 0b1110_1010
& 0b1101_1111

 0b1100_1010



Update



update_bit

- Write a function that will set bit `i` of `num` to `x`

```
#include <stdint.h>
#include <stdio.h>

int8_t update_bit(int8_t num, int8_t i, int8_t x) {

}

int main(void) {
    printf("%d\n", update_bit(0b0101, 2, 0));
    printf("%d\n", update_bit(0b0101, 1, 1));
    return 0;
}
```

Example Output

1
7



update_bit

- Write a function that will set bit `i` of `num` to `x`

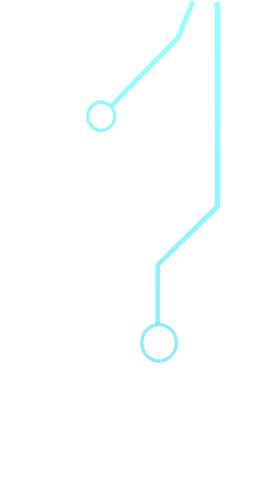
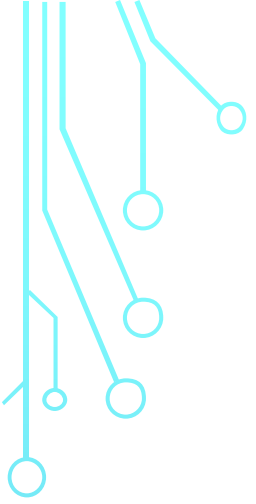
```
#include <stdint.h>
#include <stdio.h>

int8_t update_bit(int8_t num, int8_t i, int8_t x) {
    int8_t mask = ~(1 << i);
    int8_t clear_bit_i = num & mask;
    return clear_bit_i | (x << i);
}

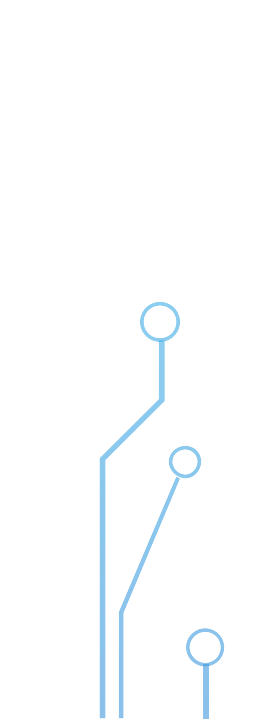
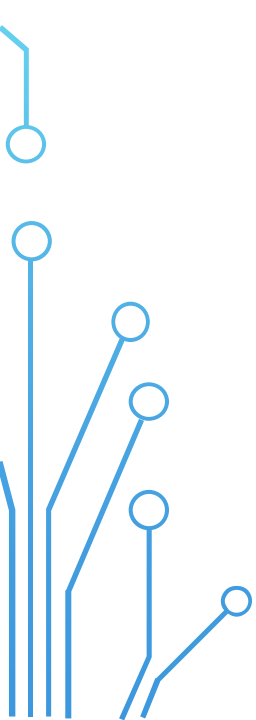
int main(void) {
    printf("%d\n", update_bit(0b0101, 2, 0));
    printf("%d\n", update_bit(0b0101, 1, 1));
    return 0;
}
```

Example Output

1
7



Applications of Bitwise Operators



Applications of Bit Manipulation

- Performance optimization in arithmetic operations
 - Bit shifting is much more efficient than multiplying and dividing
- Example: To multiply a number x by 4
 - $x *= 4$
 - $x << 2$ This is much faster!
- Used in: Embedded systems, game engines, and performance-critical code.

Performance Optimizations in Arithmetic Operations

- Using only bitwise operations and addition, write a line of C that will multiply a number n by 20.
 - Hint: What combination of powers of 2 add up to 20?

Performance Optimizations in Arithmetic Operations



- Using only bitwise operations and addition, write a line of C that will multiply a number n by 20.
 - Hint: What combination of powers of 2 add up to 20?
 - $n = 16n + 4n$
 - `n = n << 4 + n << 2`

Applications of Bit Manipulation

- Data Compression and Encoding
 - **Bit-packing:** Store multiple small values (like flags, booleans, or short integers) into a single larger data type using shifts and bitwise OR.
 - Used in: Video encoding and file compression algorithms

Bit Packing

- Write a line of code that combines two 4-bit values (nibbles) into a single `uint8_t`.
- The final `uint8_t` should be formatted as `{high_nibble, low_nibble}`
- Example:
 - `high_nibble = 0b0101`
 - `low_nibble = 0b 1100`
 - `result = {high_nibble, low_nibble} = 0b0101_1100`

`uint8_t` packed =

Bit Packing

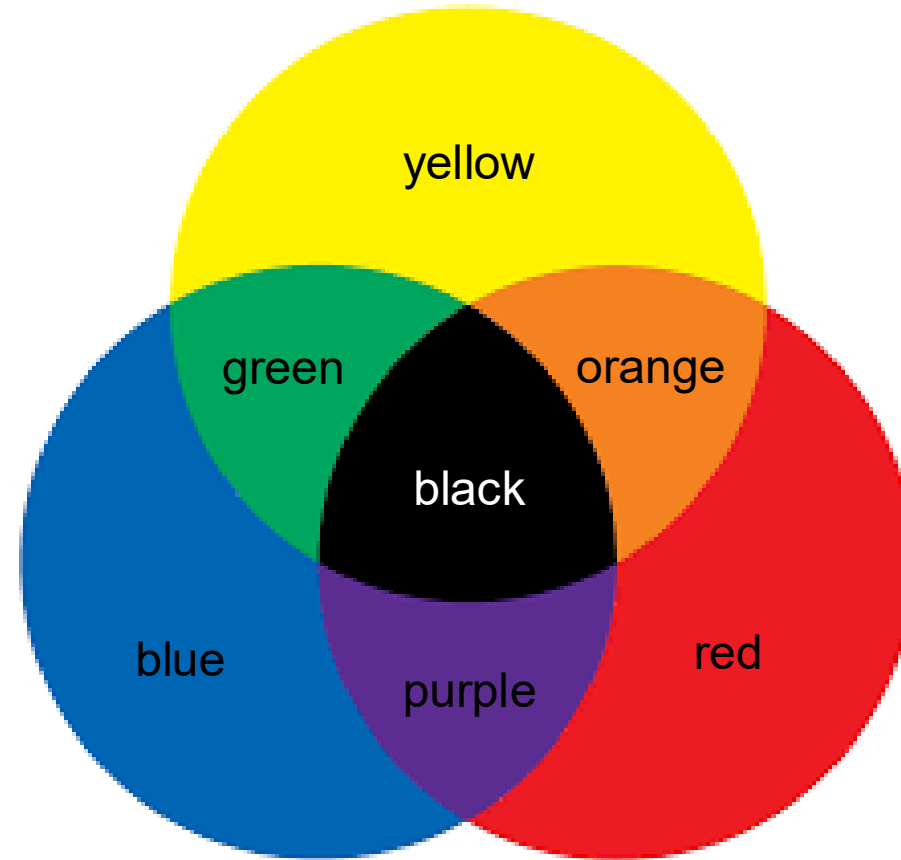


- Write a line of code that combines two 4-bit values (nibbles) into a single `uint8_t`.
- The final `uint8_t` should be formatted as `{high_nibble, low_nibble}`
- Example:
 - `high_nibble = 0b0101`
 - `low_nibble = 0b1100`
 - `result = {high_nibble, low_nibble} = 0b0101_1100`

```
uint8_t packed = (high_nibble << 4) | low_nibble;
```

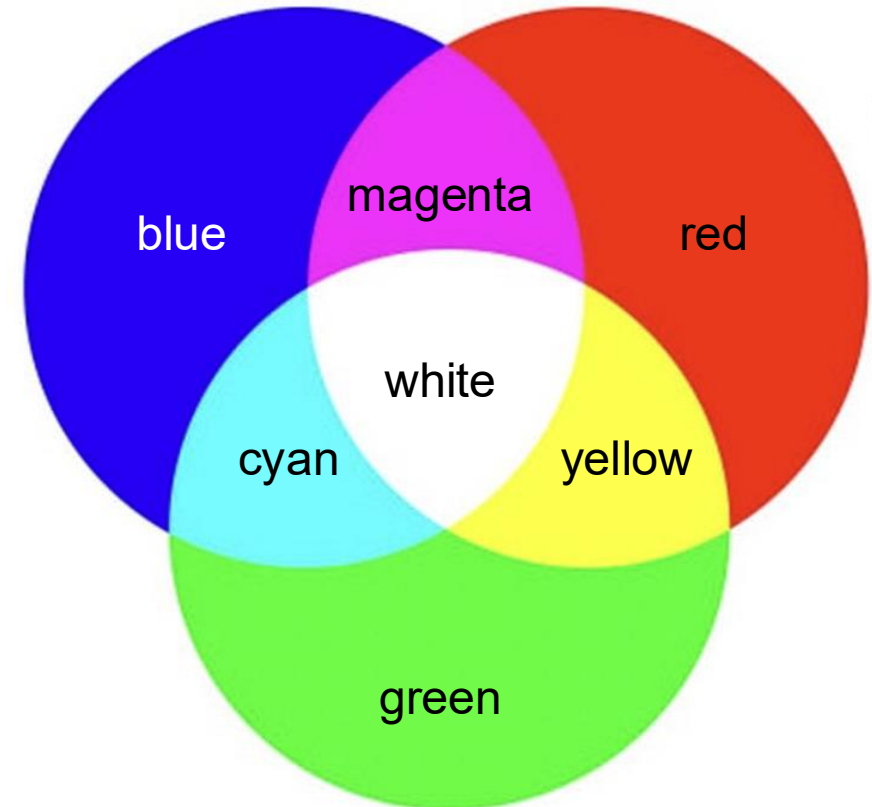
Subtractive Color Model

- Describes how pigments combine to produce colors by reflecting light
- Primary colors: yellow, blue, and red
- As color is added, the result is darker
- Absence of color: white
- All colors: black



Additive Color Model

- Describes how lights combines to produce colors
- Primary colors: red, green, and blue
- As color is added, the result becomes lighter
- Absence of color: black
- All colors: white



Additive Color Model



Complete Image



Red Channel



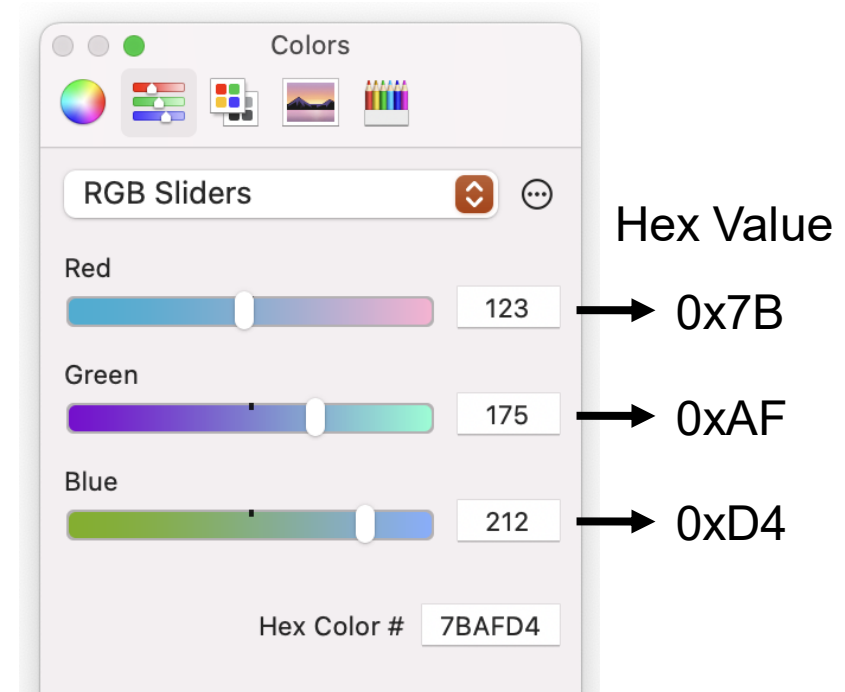
Green Channel



Blue Channel

Hex Color Codes

- The values in an RGB color code represent the intensity of each color channel. Each channel's intensity ranges from:
 - 0 (0x00) → No intensity (the channel is completely off)
 - 255 (0xFF) → Full intensity (the channel is at its maximum brightness)



Color code for Carolina Blue!

Extracting Color Channels

```
uint32_t color = 0x7BAFD4; // Example color (Hex: Carolina Blue)
```

```
// Extract Red, Green, and Blue values
```

```
uint32_t red =
```

```
uint32_t green =
```

```
uint32_t blue =
```

0x7BAFD4

Red = 0x7B

Green = 0xAF

Blue = 0xD4

Extracting Color Channels

```
uint32_t color = 0x7BAFD4; // Example color (Hex: Carolina Blue)
```

```
// Extract Red, Green, and Blue values
```

```
uint32_t red =
```

```
uint32_t green =
```

```
uint32_t blue = color & 0xFF;
```

Isolating Blue

$$\begin{array}{r} 0x7BAFD4 \\ \& 0x0000FF \\ \hline 0x0000D4 \end{array}$$

Extracting Color Channels

```
uint32_t color = 0x7BAFD4; // Example color (Hex: Carolina Blue)
```

```
// Extract Red, Green, and Blue values
```

```
uint32_t red =
```

```
uint32_t green =
```

```
uint32_t blue = color & 0xFF;
```

Isolating Blue

$$\begin{array}{r} 0x7BAFD4 \\ \& 0x0000FF \\ \hline 0x0000D4 \end{array}$$

Extracting Color Channels



```
uint32_t color = 0x7BAFD4; // Example color (Hex: Carolina Blue)
```

```
// Extract Red, Green, and Blue values
```

```
uint32_t red = (color >> 16) & 0xFF;
```

```
uint32_t green = (color >> 8) & 0xFF;
```

```
uint32_t blue = color & 0xFF;
```

Isolating Red

$0x7BAFD4 \gg 16 = 0x00007B$

$$\begin{array}{r} & 0x00007B \\ & \& 0x0000FF \\ \hline & 0x00007B \end{array}$$

Isolating Green

$0x7BAFD4 \gg 8 = 0x007BAF$

$$\begin{array}{r} & 0x007BAF \\ & \& 0x0000FF \\ \hline & 0x0000AF \end{array}$$

Isolating Blue

$$\begin{array}{r} & 0x7BAFD4 \\ & \& 0x0000FF \\ \hline & 0x0000D4 \end{array}$$